



國立清華大學
NATIONAL TSING HUA UNIVERSITY

Deep Learning and Applications

Ching-I Huang

Variational Autoencoder (VAE)

- Goal: The model should reproduce data that looks like the real thing

1. Real data samples

$$x^{(1)}, x^{(2)}, \dots, x^{(N)} \sim p(x) \quad \text{true but unknown data distribution}$$

2. Learn a generative model $p_{\theta}(x)$ that explains real data

$$\max_{\theta} \log p_{\theta}(x) \quad \text{parameters } \theta \text{ that maximize the likelihood of observed data}$$

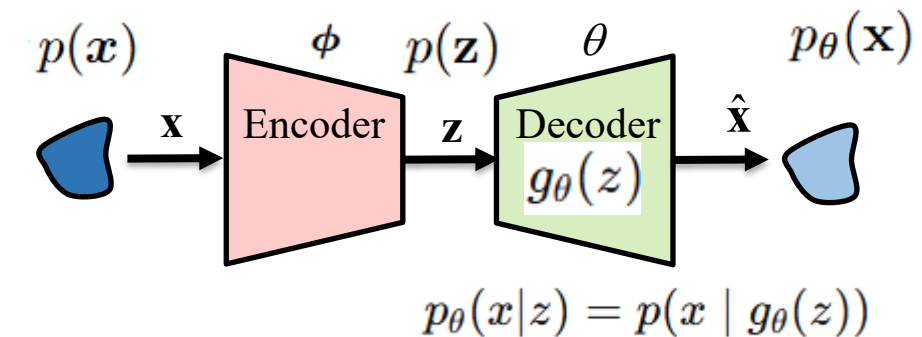
- A **generative model**

Assume data is generated from hidden causes z ,
consider *all possible latent causes* that could have produced the data x .

$$p_{\theta}(x) = \int \underbrace{p_{\theta}(x|z)}_{\text{likelihood function}} \underbrace{p(z)}_{\text{latent prior (e.g., } N(0, I))} dz = \int_{z \in \mathbb{R}^d} p_{\theta}(x | z) p(z) dz$$

intractable

1. z in high dimensions
2. $g_{\theta}(z)$ is a complicated neural network



Given a latent variable z , how likely is it to observe x under our generative model defined by $g_{\theta}(z)$



Variational Autoencoder (VAE)

- Introduce a tractable, learnable approximation

$$p(z|x) \leftarrow \underset{\text{encoder}}{q_\phi(z|x)}$$

- Goal of MLE: choose θ to make this large for data x .

1. Insert a harmless “1”

$$\log p_\theta(x) = \log p_\theta(x) \int q_\phi(z|x) dz.$$

2. Pull the constant inside the integral

$$\log p_\theta(x) = \int q_\phi(z|x) (\log p_\theta(x)) dz = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x)].$$

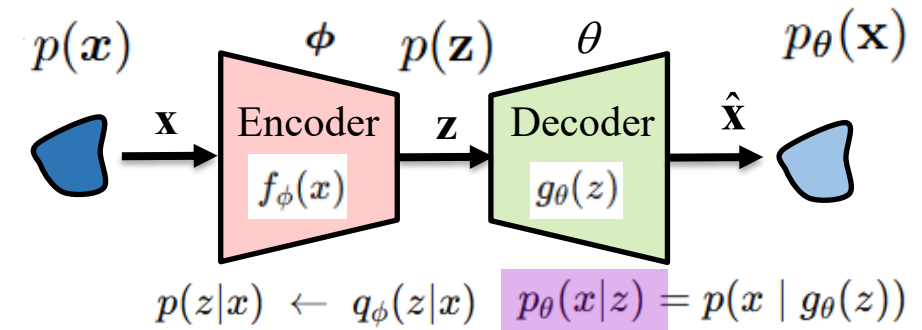
3. Replace p_θ with a ratio involving the joint and the posterior

$$\begin{aligned} \log p_\theta(x) &= \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{p_\theta(z|x)} \right] = \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z) q_\phi(z|x)}{p_\theta(z|x) q_\phi(z|x)} \right] \\ &= \mathbb{E}_{q_\phi} \left[\log \frac{p_\theta(x, z)}{q_\phi(z|x)} \right] + \mathbb{E}_{q_\phi} \left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)} \right] \end{aligned}$$

The encoder's guessed latent variable distribution

The posterior distribution implied by the decoder model

The encoder is learning to find the right z that makes sense of x .





Variational Autoencoder (VAE)

- **Kullback–Leibler divergence**, a measure of how different two probability distributions are.

$$D_{KL}(q_\phi(z|x) || p_\theta(z|x)) = \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)} \right]$$

If they are identical $\rightarrow \text{KL} = 0$
If they differ $\rightarrow \text{KL} > 0$

The more q_ϕ diverges from the true posterior, the worse the bound becomes.

- KL term is intractable

$$D_{KL}(q_\phi(z|x) || p_\theta(z|x)) = \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)} \right] = \mathbb{E}_{q_\phi(z|x)} \left[\log q_\phi(z|x) - \log p_\theta(x|z) - \log p(z) + \log p_\theta(x) \right]$$

intractable

$$\log p_\theta(x) = \underbrace{\mathbb{E}_{q_\phi} [\log p_\theta(x, z) - \log q_\phi(z | x)]}_{\mathcal{L}_{ELBO}(\theta, \phi)} + \underbrace{D_{KL}(q_\phi(z | x) || p_\theta(z | x))}_{\text{KL divergence } \geq 0, \text{ but intractable.}}$$

$\log p_\theta(x) \geq \mathcal{L}_{ELBO}(\theta, \phi)$

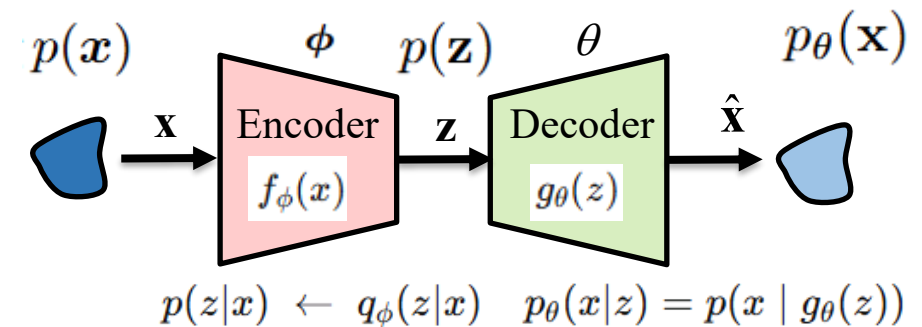
$\rightarrow$ maximizing the ELBO $\max_{\theta, \phi} \mathcal{L}_{ELBO}(\theta, \phi)$ indirectly minimizes that KL gap



Variational Autoencoder (VAE)

- Evidence lower bound (ELBO) on data likelihood

$$\begin{aligned}\mathcal{L}_{\text{ELBO}}(\theta, \phi) &= \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p(x, z)}{q_{\phi}(z | x)} \right] = \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)} \right] \\ &= \mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x | z)] + \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p(z)}{q_{\phi}(z | x)} \right] \\ &= \underbrace{\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x | z)]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_{\phi}(z | x) \| p(z))}_{\text{prior matching term}}\end{aligned}$$



- Reconstruction Term
 - Measures **how well the decoder** can reconstruct the observed data x when the latent variable z is sampled from the encoder $q_{\phi}(z|x)$.
 - Maximizing** this term encourages realistic outputs resembling the true input (data likelihood).
- Prior Matching Term
 - Measures **how far the encoder's posterior** $q_{\phi}(z|x)$ deviates from the prior $p(z) = \mathcal{N}(0, I)$
 - The indirect regularization from the KL term prevents the encoder from trivially copying x to the decoder.

Variational Autoencoder (VAE)

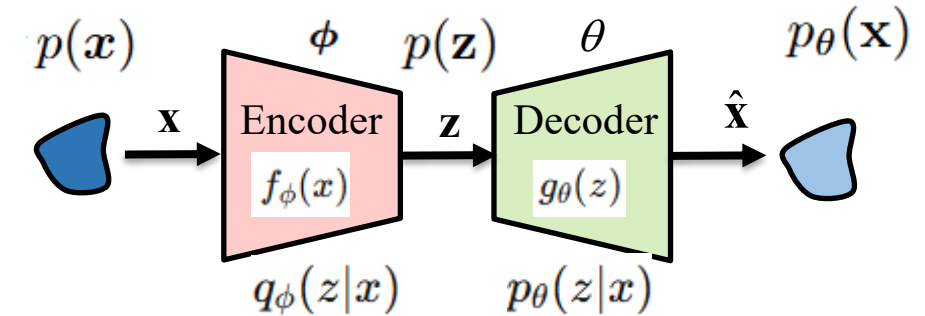
- Maximum likelihood objective

$$\log p_{\theta}(x) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x)||p(z)) + D_{KL}(q_{\phi}(z|x)||p_{\theta}(z|x))$$

The decoder learns to reconstruct the data

Prevents the encoder from mapping each data point to its own isolated region in latent space.

The encoder is learning to find the right z that makes sense of x .



- Training Dynamics (Likelihood Trend)
 - ELBO gradually increases as encoder/decoder start coordinating.
 - KL divergence shrinks — encoder approaches true posterior.
 - ELBO continues improving; log-likelihood typically rises as the model better fits the data.

Log-likelihood may not increase every epoch (not monotonic).

The overall trend of increasing ELBO → higher likelihood over time.

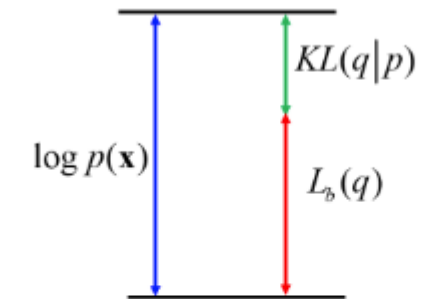


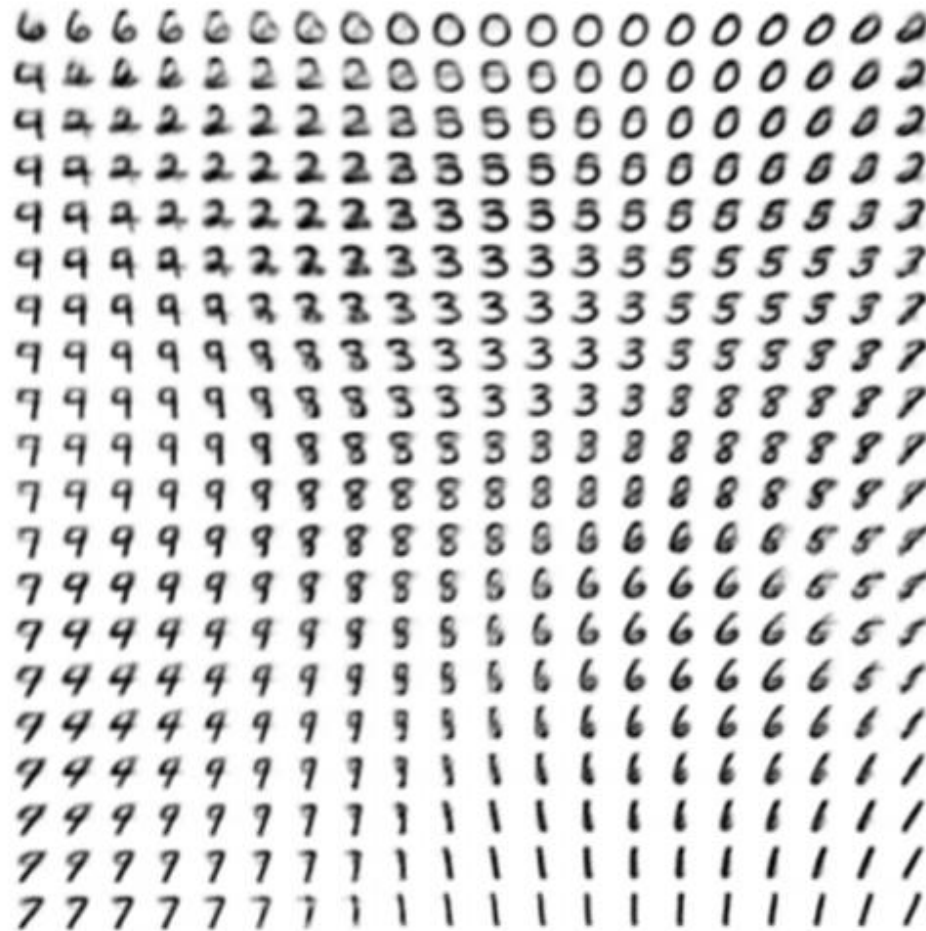
Fig. 18: The distribution $q(z)$ is held fixed, and the lower bound L_b is maximized with respect to the parameter vector ϕ , yielding a revised value ϕ_{new} . Since the KL divergence is nonnegative, this process causes the log-likelihood $\log p(x|\phi)$ to increase by at least as much as the lower bound does.

Backward pass

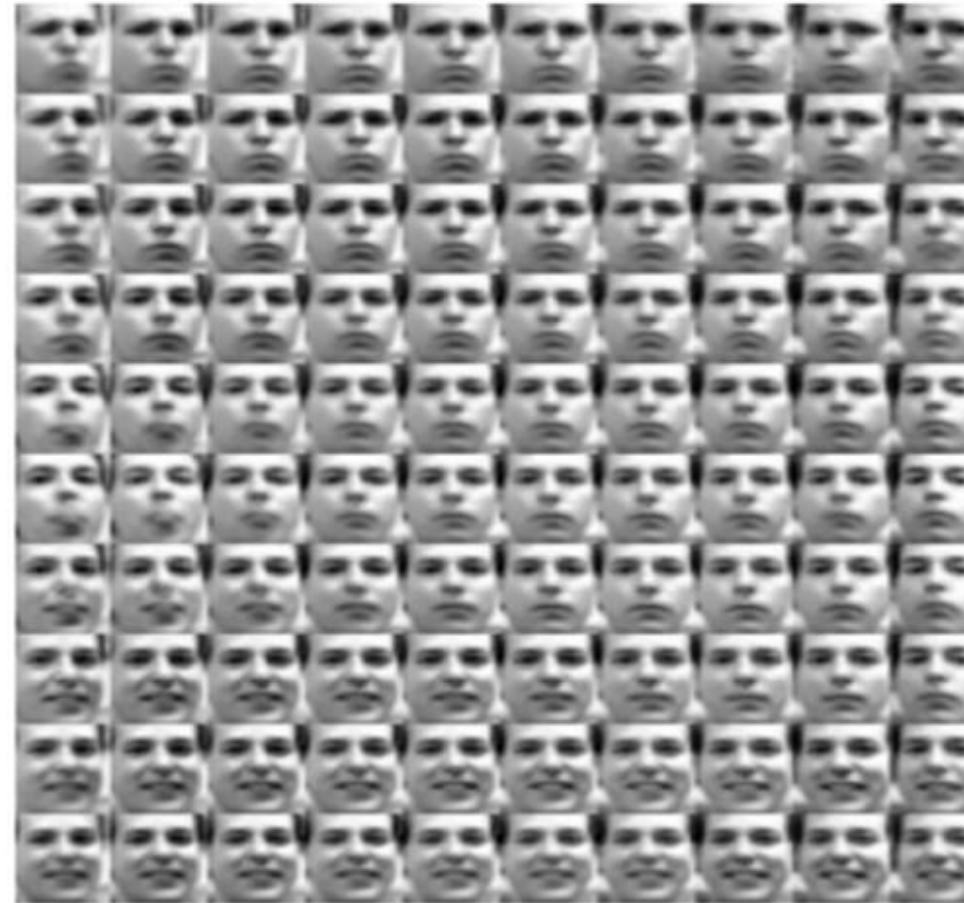
- Update encoder parameters ϕ (fix θ)
- Update decoder parameters θ (fix ϕ)



Variational Autoencoder (VAE)



(a) The two-dimensional map of the MNIST manifold

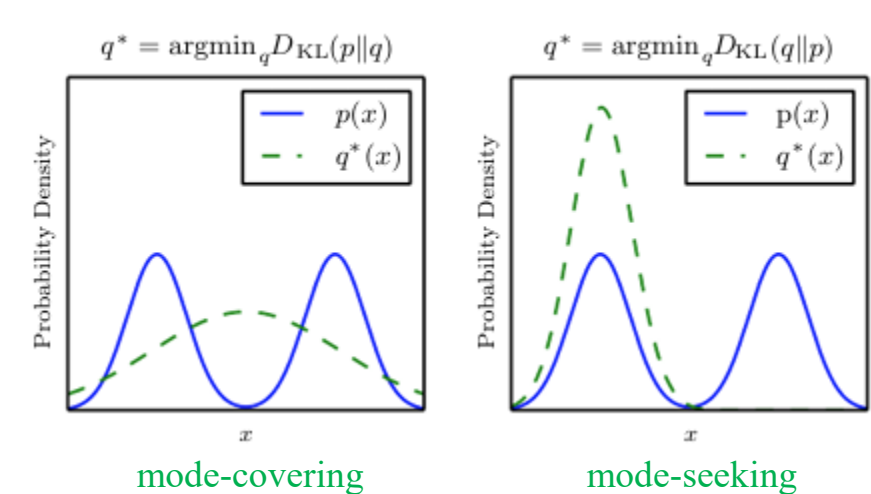


(b) The two-dimensional map of the Frey faces manifold

Fig. 19: An understanding of how the model works by training a model with a 2-D latent code, even if we believe the intrinsic dimensionality of the data manifold is much higher. The images shown are not examples from the training set but images actually generated by the model $p(x|z)$, simply by changing the 2-D “code” z (each image corresponds to a different choice of “code” z on a 2-D uniform grid).

Variational Autoencoder (VAE)

- The VAE effectively compresses high-dimensional image data into a smooth, low-dimensional manifold.
- In this **latent space**, similar images are positioned close to each other, and continuous variations in the latent vector $\mathbf{z}$ leads to **smooth transitions** in the generated images.
- To maintain this structure, the **encoder** is constrained by the **Gaussian prior**, resulting in a **trade-off between reconstruction fidelity and smoothness**.
- This limitation arises from the **likelihood assumption** used during training: when multiple valid reconstructions exist, the model tends to produce their **average**, leading to **blurred outputs**.
- The **overall structure** of the data often appear **blurry** or **low in detail**.





Generative Adversarial Network

- Goal: Generate sharper, more realistic images.

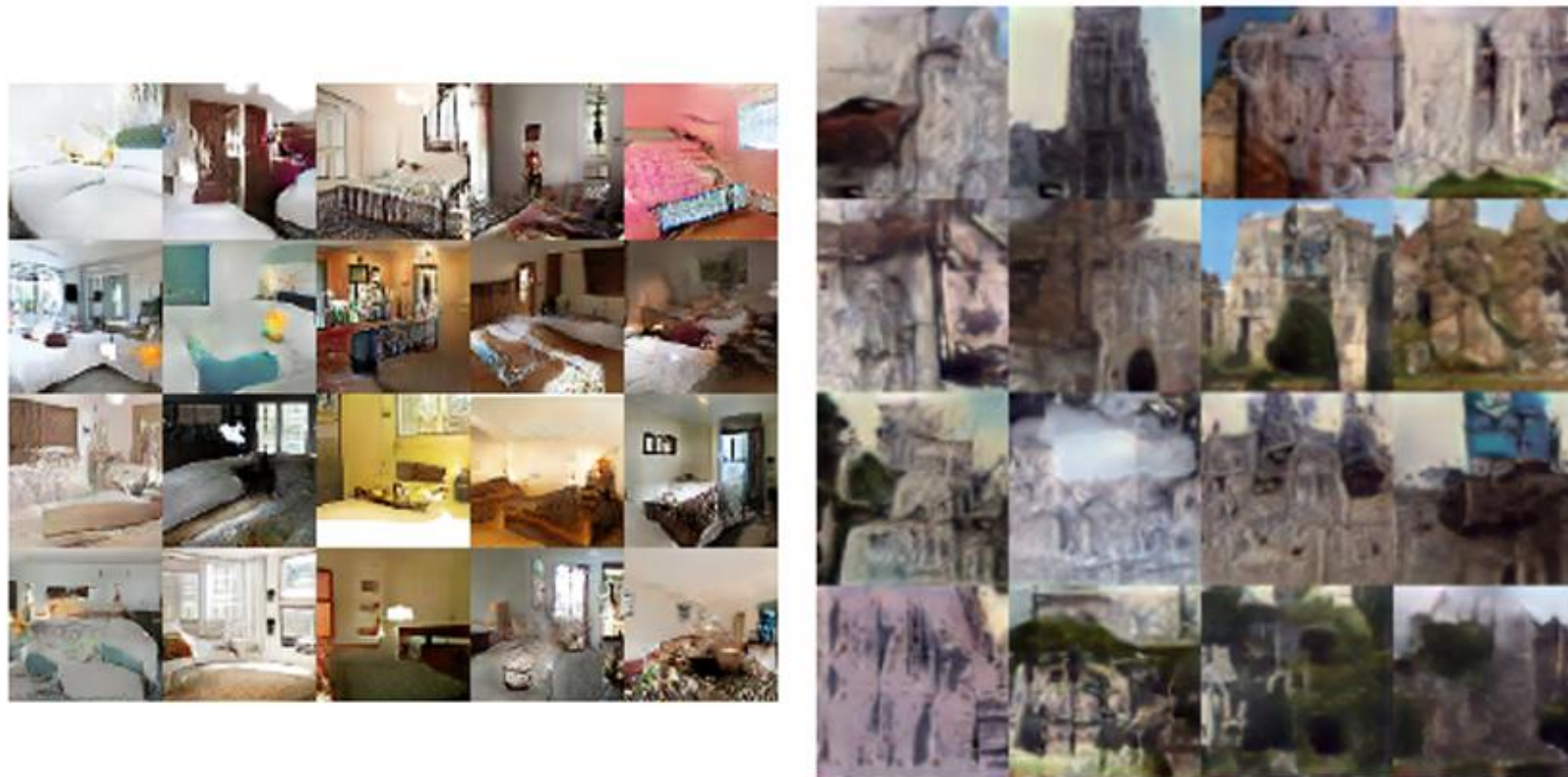


Figure 20.7: Images generated by GANs trained on the LSUN dataset. (*Left*) Images of bedrooms generated by a DCGAN model, reproduced with permission from [Radford et al. \(2015\)](#). (*Right*) Images of churches generated by a LAPGAN model, reproduced with permission from [Denton et al. \(2015\)](#).



Generative Adversarial Network

- In VAEs, the model $\nabla_{\theta} \log P_{\theta}(x)$ assumes a Gaussian likelihood — when multiple reconstructions are possible, it averages them, producing blurry results.
- **GANs are formulated as a game-theoretic framework** in which the generator network competes against an adversary, the discriminator.

- GANs consist of two competing neural networks:

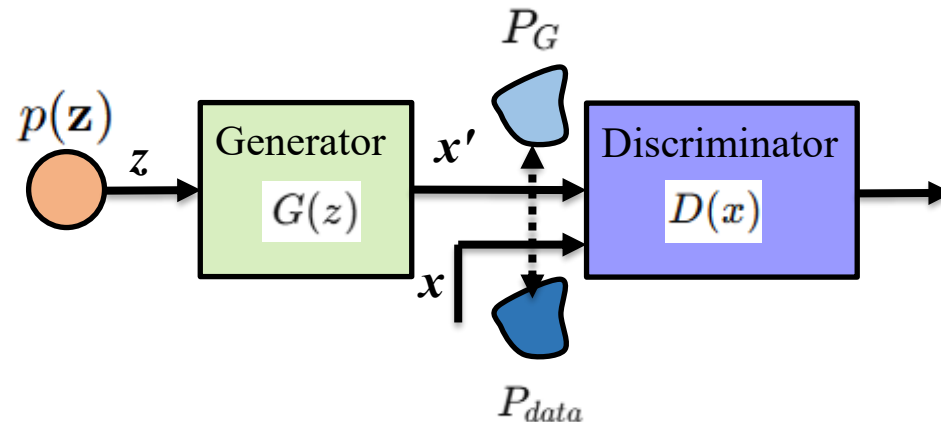
1. **Generator (G):** Generates fake data from random noise

$$x' = G(z)$$

2. **Discriminator (D):** Distinguishes real samples from generated ones.

$$D(x) = \begin{cases} 1, & x \text{ is real (from } P_{data}) \\ 0, & x = G(z) \text{ (from } P_G) \end{cases}$$

distribution of fake data induced by G



$$z \sim p(z) \Rightarrow x' = G(z) \Rightarrow x' \sim P_G \Rightarrow P_G \Longleftrightarrow P_{data}$$

GAN類神經網路

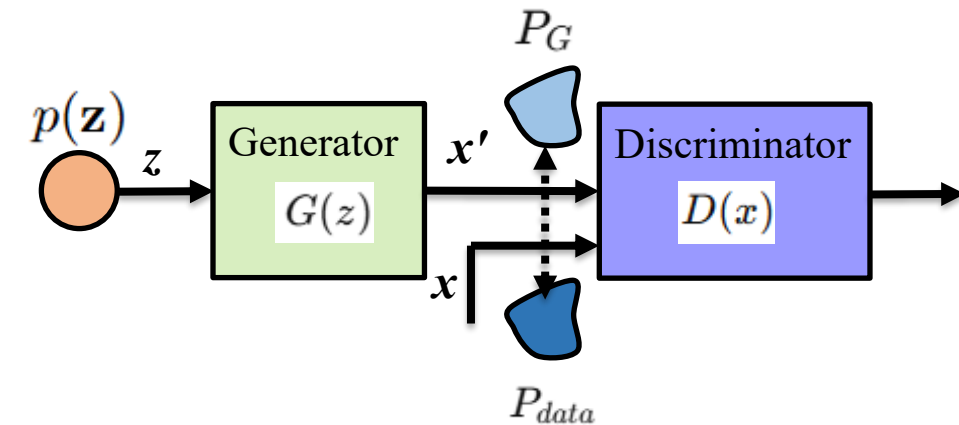
繪製：Yoshi Liao





Generative Adversarial Network

- Goal: $z \sim p(z) \Rightarrow x' = G(z) \Rightarrow x' \sim P_G \Rightarrow P_G \iff P_{data}$
- Generator



- $G^* = \arg \min_G \text{Div}(P_G, P_{data})$ unknown
- Approximate the divergence by discriminator

$$V(G, D) = \mathbb{E}_{x \sim P_{data}} [\log D(x)] + \mathbb{E}_{x \sim P_G} [\log(1 - D(x))]$$

- Minimax Optimization

$$D^* = \arg \max_D V(G, D)$$

maximize $V(G, D) \rightarrow$ classify real vs. fake correctly.

an **adversarial game** between G and D .

$$G^* = \arg \min_G \max_D V(G, D)$$

minimize $V(G, D) \rightarrow$ fool D by making fake data look real.

$$\begin{aligned} \arg \min_{\Theta_G} \max_{\Theta_D} C(\Theta_G, \Theta_D) &= \arg \min_{\Theta_G} \max_{\Theta_D} \left[\sum_n \log D(x^{(n)}) + \sum_m \log(1 - D(G(z^{(m)}))) \right] \\ &\quad \text{fool the discriminator} \quad \text{distinguish real vs. fake} \\ &= \arg \min_{\Theta_G} \max_{\Theta_D} \left[\sum_n \log \hat{\rho}^{(n)} + \sum_m \log(1 - \hat{\rho}^{(m)}) \right] \quad \text{binary classifier} \\ &\quad \parallel \\ &\quad \log P(\mathbf{X}|\Theta) \end{aligned}$$

Equivalent to training a binary classifier with cross-entropy loss.

log-likelihood learning



Generative Adversarial Network

$$\arg \min_{\Theta_G} \max_{\Theta_D} C(\Theta_G, \Theta_D) = \arg \min_{\Theta_G} \max_{\Theta_D} \left[\underbrace{\sum_n \log D(x^{(n)}) + \sum_m \log(1 - D(G(z^{(m)})))}_{\text{fake part}} \right]$$

- Training Algorithm of GANs

1. Initialize the parameters: Θ_G for the Generator, Θ_D for the Discriminator.
2. At each iteration (one SGD step):
 - a. Fix Generator G , update Discriminator D

$$L_D = -[\log D(x_{real}) + \log(1 - D(G(z)))] \quad \text{minimizing cross-entropy}$$

$$\Theta_D \leftarrow \Theta_D + \eta \nabla_{\Theta_D} \left[\sum_n \log D(x^{(n)}) + \sum_m \log(1 - D(G(z^{(m)}))) \right]$$

Repeat K times (to make D strong enough before updating G)

- b. Fix Discriminator D , update Generator G

$$L_G = -\mathbb{E}_{z \sim p(z)} [\log D(G(z))] \quad \text{The generator learns indirectly through } D \text{'s gradients.}$$

$$\Theta_G \leftarrow \Theta_G - \eta \nabla_{\Theta_G} \left[\sum_m \log(1 - D(G(z^{(m)}))) \right]$$

Execute once



Generative Adversarial Network

- Goal: $z \sim p(z) \Rightarrow x' = G(z) \Rightarrow x' \sim P_G \Rightarrow P_G \iff P_{data}$
- Generator

$$G^* = \arg \min_G \text{Div}(P_G, P_{data}) \text{ unknown}$$

- Approximate the divergence by discriminator

$$V(G, D) = \mathbb{E}_{x \sim P_{data}} [\log D(x)] + \mathbb{E}_{x \sim P_G} [\log(1 - D(x))]$$

- Discriminator's objective

$$D^* = \arg \max_D V(G, D) = \max_{\Theta_D} \mathbb{E}_{x \sim P_{data}} [\log D(x)] + \mathbb{E}_{x \sim P_G} [\log(1 - D(x))]$$

Substitute expectation with integrals

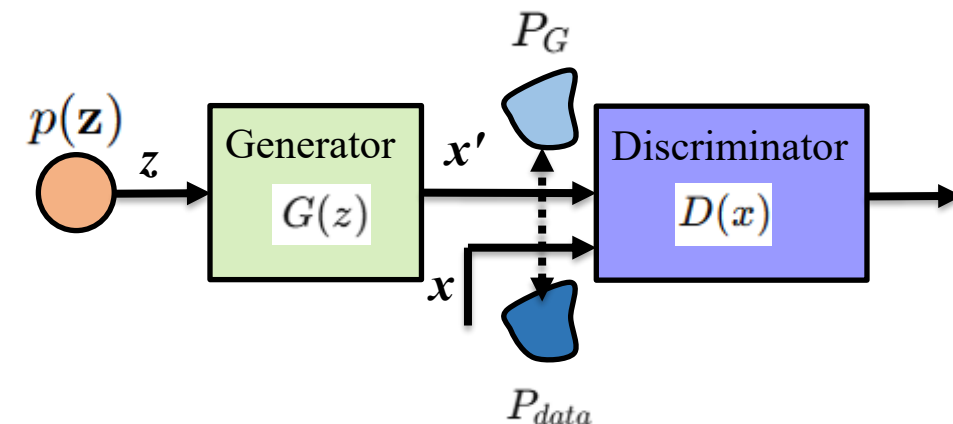
$$= \max_{\Theta_D} \int_x [P_{data}(x) \log D(x) + P_G(x) \log(1 - D(x))] dx$$

Find the optimal discriminator $D^*(x)$

$$\frac{\partial}{\partial D(x)} [P_{data}(x) \log D(x) + P_G(x) \log(1 - D(x))] = 0 \Rightarrow D^*(x) = \frac{P_{data}(x)}{P_{data}(x) + P_G(x)} \in [0, 1]$$

$D^*(x)=0.5$: real and fake are indistinguishable

$$\Rightarrow V(G, D^*) = \int_x \left[P_{data}(x) \log \frac{P_{data}(x)}{P_{data}(x) + P_G(x)} + P_G(x) \log \frac{P_G(x)}{P_{data}(x) + P_G(x)} \right] dx = -2 \log 2 + 2 D_{JS}(P_{data} \parallel P_G)$$



$$\frac{P_{data}(x)}{D(x)} - \frac{P_G(x)}{1 - D(x)} = 0.$$

$$P_{data}(x)(1 - D(x)) = P_G(x)D(x)$$

$$P_{data}(x) - P_{data}(x)D(x) = P_G(x)D(x)$$

$$D(x)(P_{data}(x) + P_G(x)) = P_{data}(x)$$

Jensen–Shannon Divergence (JSD)
similarity between two distributions
 $D_{JS}(P_{data} \parallel P_G)$

$$= \frac{1}{2} D_{KL}(P_{data} \parallel M) + \frac{1}{2} D_{KL}(P_G \parallel M)$$

$$M = \frac{1}{2}(P_{data} + P_G)$$



Generative Adversarial Network

- Jensen–Shannon Divergence Saturation Problem

$$\min_G \max_D V(G, D) = -2 \log 2 + 2D_{JS}(P_{data} || P_G)$$

the generator G implicitly minimizes the divergence between the real data distribution P_{data} and the generated data distribution P_G .

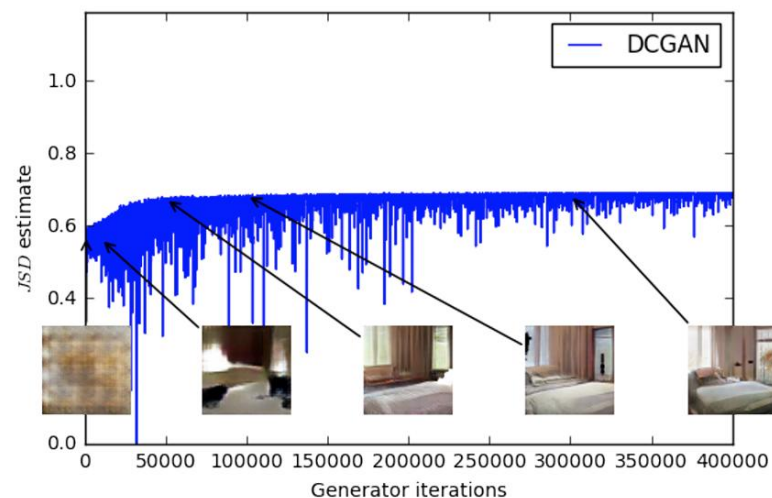
At the early stage of training, P_G and P_{data} are almost **non-overlapping**



$$D_{JS}(P_{data} || P_G) \approx \log 2 \approx 0.693$$

and the discriminator easily distinguishes real and fake samples: $D(x_{real}) \approx 1$, $D(G(z)) \approx 0$.

As a result, the generator receives almost no gradient: $\nabla_{\theta_G} \log(1 - D(G(z))) \approx 0$. [vanishing gradient problem \(stop learning\)](#)



Note also that the JS estimate usually stays constant or goes up instead of going down. In fact it often remains very close to $\log 2 \approx 0.69$ which is the highest value taken by the JS distance. In other words, the JS distance saturates, the discriminator has zero loss, and the generated samples are in some cases meaningful (DCGAN generator, top right plot) and in other cases collapse to a single nonsensical image (Goodfellow et al., 2014)